

Introduction to Data Science — Final Exam Paper 6 (Extreme Difficult)

Course Code: BCSE2C05 | Semester: II | Max Marks: 100 | Time: 3 Hours Difficulty: ★★★★★ Extreme Difficult

SECTION A — Short Answer (5 Marks Each, Do Any 4 of 5 = 20 Marks)

Q1. Given symmetric and asymmetric binary attribute vectors, compute both Simple Matching Coefficient and Jaccard Coefficient.

Given two objects X and Y with 8 binary attributes:

Attribute	A1	A2	A3	A4	A5	A6	A7	A8
X	1	0	1	1	0	0	1	0
Y	1	1	1	0	0	1	1	0

Contingency table:

	Y=1	Y=0
X=1	$f_{11} = 3$ (A1,A3,A7)	$f_{10} = 1$ (A4)
X=0	$f_{01} = 2$ (A2,A6)	$f_{00} = 2$ (A5,A8)

Simple Matching Coefficient (Symmetric):

$$SMC = (f_{11} + f_{00}) / (f_{11} + f_{10} + f_{01} + f_{00}) = (3 + 2) / (3 + 1 + 2 + 2) = 5/8 = 0.625$$

Jaccard Coefficient (Asymmetric):

$$J = f_{11} / (f_{11} + f_{10} + f_{01}) = 3 / (3 + 1 + 2) = 3/6 = 0.5$$

SMC counts 0-0 matches as agreement; Jaccard ignores them (useful when 0-0 is uninteresting, like absence of rare disease).

Q2. Compute the Z-Score normalization for a dataset and identify outliers using the Z-Score method.

Data: {45, 50, 55, 60, 95}

Step 1: Mean (μ) = $(45+50+55+60+95)/5 = 305/5 = 61$

Step 2: Standard Deviation (σ):

- $\Sigma(x_i-\mu)^2 = (-16)^2+(-11)^2+(-6)^2+(-1)^2+(34)^2 = 256+121+36+1+1156 = 1570$
- $\sigma = \sqrt{(1570/5)} = \sqrt{314} = 17.72$

Step 3: Z-Scores:

X	$Z = (X-\mu)/\sigma$	Outlier?
45	$(45-61)/17.72 = -0.90$	No
50	$(50-61)/17.72 = -0.62$	No
55	$(55-61)/17.72 = -0.34$	No
60	$(60-61)/17.72 = -0.06$	No
95	$(95-61)/17.72 = +1.92$	No (threshold is ± 3)

With stricter threshold (± 2): 95 would be borderline. With ± 3 standard threshold: no outliers in this set.

Q3. Write Python code to perform label encoding and one-hot encoding on categorical data.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({
    'City': ['Delhi', 'Mumbai', 'Pune', 'Delhi', 'Mumbai'],
    'Size': ['Small', 'Medium', 'Large', 'Medium', 'Small']
})

# — Label Encoding —
le = LabelEncoder()
df['City_Encoded'] = le.fit_transform(df['City'])
# Delhi→0, Mumbai→1, Pune→2

df['Size_Encoded'] = le.fit_transform(df['Size'])
# Large→0, Medium→1, Small→2

print("Label Encoded:\n", df)

# — One-Hot Encoding —
df_onehot = pd.get_dummies(df[['City', 'Size']], dtype=int)
print("\nOne-Hot Encoded:\n", df_onehot)
```

Output (One-Hot):

	City_Delhi	City_Mumbai	City_Pune	Size_Large	Size_Medium	Size_Small
0	1	0	0	0	0	1
1	0	1	0	0	1	0
2	0	0	1	1	0	0
3	1	0	0	0	1	0
4	0	1	0	0	0	1

When to use:

- **Label Encoding:** When categories have ordinal relationship (Small < Medium < Large).
- **One-Hot Encoding:** When categories have no order (Delhi, Mumbai, Pune).

Q4. Explain the concept of data discretization. Perform equal-width binning on a given dataset.

Data Discretization converts continuous attributes into categorical (discrete) intervals/bins. It reduces data complexity and noise.

Types:

- **Equal-Width Binning:** Divide range into bins of equal size.
- **Equal-Frequency (Depth) Binning:** Each bin has the same number of data points.

Equal-Width Binning Example:

Data (sorted): {5, 10, 15, 20, 25, 30, 35, 40, 45}

- Range = 45 – 5 = 40
- Number of bins = 3
- Bin width = 40/3 ≈ 13.3

Bin	Range	Data Points	Smoothed (bin mean)
Bin 1	[5, 18.3)	5, 10, 15	10
Bin 2	[18.3, 31.6)	20, 25, 30	25
Bin 3	[31.6, 45]	35, 40, 45	40

Smoothing by bin means replaces each value with the average of its bin, reducing noise.

Q5. Explain the concept of data drift and concept drift. Why are they relevant in deployed ML models?

Data Drift (Covariate Shift):

- The distribution of **input features** changes over time while the true relationship between features and target remains the same.
- **Example:** A loan approval model trained on pre-COVID data sees a surge of applications from new demographics (gig workers) post-COVID — feature distributions shift.

Concept Drift:

- The **underlying relationship** between input features and target variable changes over time.
- **Example:** A product recommendation model — customer preferences change seasonally (winter clothing interests spike in November but not in June).

Why Relevant:

1. Model performance degrades silently without monitoring.
2. Predictions become unreliable and potentially harmful.
3. Requires retraining or model update strategies.
4. Monitored using statistical tests (KS test, PSI score) and dashboards.

SECTION B — Analytical Questions (8 Marks Each, Do Any 5 of 6 = 40 Marks)

Q6. Given a confusion matrix, calculate Accuracy, Precision, Recall, F1-Score, and Specificity. Analyse which metric matters more for a medical diagnosis scenario.

Confusion Matrix for a disease prediction model:

	Predicted Positive	Predicted Negative
Actual Positive	TP = 80	FN = 20
Actual Negative	FP = 30	TN = 870

Calculations:

Metric	Formula	Value
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	$(80+870)/1000 = 95.0\%$
Precision	$TP/(TP+FP)$	$80/110 = 72.7\%$
Recall (Sensitivity)	$TP/(TP+FN)$	$80/100 = 80.0\%$
Specificity	$TN/(TN+FP)$	$870/900 = 96.7\%$
F1-Score	$2 \times (P \times R) / (P + R)$	$2 \times (0.727 \times 0.80) / (0.727 + 0.80) = 76.2\%$

Analysis for Medical Diagnosis:

In medical scenarios, **Recall is the most critical metric** because:

- FN (False Negative) = Telling a sick patient they're healthy → **life-threatening**.
- FP (False Positive) = Telling a healthy patient they're sick → further tests, but not dangerous.
- Missing 20 out of 100 actual positives means **20% of patients with the disease go undiagnosed**.

Even though accuracy is 95%, the 20% miss rate (FN) is unacceptable. A model should maximize recall even at the cost of precision (more false alarms are preferable to missed cases).

Q7. Write an advanced Pandas program that performs multi-level grouping, pivot tables, and conditional aggregation.

```

import pandas as pd
import numpy as np

# Create dataset
np.random.seed(42)
n = 200
data = {
    'Employee': [f'Emp_{i}' for i in range(1, n+1)],
    'Department': np.random.choice(['CSE', 'ECE', 'ME', 'CE'], n),
    'Gender': np.random.choice(['Male', 'Female'], n),
    'Experience': np.random.randint(1, 20, n),
    'Salary': np.random.normal(55000, 15000, n).astype(int),
    'Rating': np.round(np.random.uniform(2.0, 5.0, n), 1)
}
df = pd.DataFrame(data)

# — 1. Multi-level Grouping —
multi_group = df.groupby(['Department', 'Gender']).agg(
    Avg_Salary=('Salary', 'mean'),
    Max_Salary=('Salary', 'max'),
    Avg_Rating=('Rating', 'mean'),
    Count=('Employee', 'count')
).round(2)
print("Multi-level Group:\n", multi_group)

# — 2. Pivot Table —
pivot = df.pivot_table(
    values='Salary',
    index='Department',
    columns='Gender',
    aggfunc=['mean', 'count']
).round(0)
print("\nPivot Table:\n", pivot)

# — 3. Conditional Aggregation —
# Average salary of employees with Rating > 4.0 per department
high_performers = df[df['Rating'] > 4.0].groupby('Department')['Salary'].mean().round(2)
print("\nAvg Salary of High Performers (Rating > 4.0):")
print(high_performers)

# — 4. Cross-tabulation —
crosstab = pd.crosstab(df['Department'], df['Gender'], margins=True)
print("\nCross-tabulation:\n", crosstab)

# — 5. Rolling/Window Aggregation (simulated time-series) —
df_sorted = df.sort_values('Experience')
df_sorted['Rolling_Avg_Salary'] = df_sorted['Salary'].rolling(window=10).mean()
print("\nRolling Average Salary (window=10):")
print(df_sorted[['Experience', 'Salary', 'Rolling_Avg_Salary']].head(15))

# — 6. Apply custom function —
def salary_grade(salary):
    if salary > 70000: return 'A'
    elif salary > 55000: return 'B'
    elif salary > 40000: return 'C'
    else: return 'D'

df['Grade'] = df['Salary'].apply(salary_grade)

```

```
grade_distribution = df.groupby(['Department', 'Grade']).size().unstack(fill_value=0)
print("\nGrade Distribution per Department:\n", grade_distribution)
```

Q8. Critically analyse how class imbalance in datasets affects ML model performance. Discuss at least four techniques to handle imbalanced data.

Problem: In many real-world datasets, one class dominates (e.g., 95% non-fraud, 5% fraud). A model can achieve 95% accuracy by predicting everything as non-fraud — but it's useless.

Effects of Class Imbalance:

- 1. **Misleading accuracy** — High accuracy but poor detection of minority class.
- 2. **Biased learning** — Model overfits to majority class.
- 3. **Poor precision/recall** — Minority class has very low recall.
- 4. **Unstable cross-validation** — Folds may lack minority examples.

Techniques to Handle:

Technique	Type	Description
1. SMOTE	Oversampling	Generates synthetic minority samples by interpolating between nearest neighbours. Does not simply duplicate — creates new, plausible examples.
2. Random Undersampling	Undersampling	Randomly removes majority class samples. Risk: information loss.
3. Class Weights	Algorithmic	Assign higher penalty for misclassifying minority class. Most sklearn classifiers support <code>class_weight='balanced'</code> .
4. Ensemble Methods	Algorithmic	Use <code>BalancedRandomForest</code> or <code>EasyEnsemble</code> that combine undersampling with ensemble learning.
5. Anomaly Detection	Reformulation	Treat minority class as anomalies — models like <code>Isolation Forest</code> or <code>One-Class SVM</code> .
6. Threshold Tuning	Post-processing	Adjust classification threshold (e.g., from 0.5 to 0.3) to favour minority class predictions.

Evaluation for Imbalanced Data:

- Use **F1-Score, Precision, Recall, AUC-ROC** instead of accuracy.
- Use **Stratified K-Fold** cross-validation to preserve class ratio.
- Plot **Precision-Recall Curve** (more informative than ROC for imbalanced data).

Q9. Perform complete outlier detection using IQR method AND Z-Score method on a given dataset. Compare results.

Dataset: Monthly salaries = {25000, 28000, 30000, 32000, 35000, 38000, 40000, 42000, 45000, 120000}

Method 1: IQR Method

Sorted data: 25000, 28000, 30000, 32000, 35000, 38000, 40000, 42000, 45000, 120000

- Q1 (25th percentile) = 30000
- Q3 (75th percentile) = 42000
- IQR = 42000 – 30000 = **12000**
- Lower bound = 30000 – 1.5 × 12000 = **12000**
- Upper bound = 42000 + 1.5 × 12000 = **60000**

Outliers (IQR): 120000 > 60000 → **{120000} is an outlier**

Method 2: Z-Score Method

- Mean (μ) = (25000+28000+...+120000)/10 = 435000/10 = **43500**

• $\sigma = \sqrt{(\sum(x_i - \mu)^2)/N}$

x	x-μ	(x-μ) ²
25000	-18500	342,250,000
28000	-15500	240,250,000
30000	-13500	182,250,000
32000	-11500	132,250,000
35000	-8500	72,250,000
38000	-5500	30,250,000
40000	-3500	12,250,000
42000	-1500	2,250,000
45000	1500	2,250,000
120000	76500	5,852,250,000

$\sum(x-\mu)^2 = 6,868,500,000 \rightarrow \sigma = \sqrt{(686,850,000)} \approx \mathbf{26,208}$

Z-Scores:

| x | Z | Outlier? (|Z| > 3) | |:---:|:---:|:---:| | 25000 | -0.71 | No | | 120000 | +2.92 | No (close but < 3) |

Using Z > 2: **120000 is an outlier**. Using strict Z > 3: borderline.

Comparison:

Aspect	IQR Method	Z-Score Method
Detected 120000?	✔ Yes (clearly > 60000)	⚠ Borderline (Z=2.92)
Robust to outliers?	Yes (Q1/Q3 are robust)	No (mean/std are pulled by 120000)
Best for	Skewed data, non-normal distributions	Approximately normal distributions

IQR is more robust because Q1 and Q3 are not affected by extreme values, whereas mean and standard deviation are distorted by the outlier itself.

Q10. Design a complete data preprocessing pipeline for a messy real-world dataset with code.

Scenario: Raw e-commerce dataset with multiple data quality issues.

```
import pandas as pd
import numpy as np

# — Simulate messy data —
data = {
    'Customer_ID': [101, 102, 102, 103, 104, 105, 106, 107, 108, 109],
    'Name': ['Alice', 'bob', 'Bob', 'CHARLIE', 'Diana', None, 'Eve', 'Frank', 'Grace', 'Henry'],
    'Age': [25, 300, 30, -5, 28, 22, np.nan, 45, 19, 55],
    'Gender': ['F', 'M', 'Male', 'M', 'Female', 'F', 'F', 'm', 'Female', 'M'],
    'Purchase_Amount': [1500, 2000, 2000, np.nan, 5000, 800, 300, np.nan, 9999999, 4500],
    'Email': ['alice@g.com', 'bob@g.com', 'bob@g.com', 'charlie@', None, 'eve@g.com',
              'eve@g.com', 'frank@g.com', 'grace@g.com', 'henry@g.com']
}

df = pd.DataFrame(data)
print("Original:\n", df)
```

```
# — Step 1: Remove Duplicates —
df = df.drop_duplicates(subset='Customer_ID', keep='first')
print(f"\nAfter dedup: {len(df)} records")

# — Step 2: Standardize Text —
df['Name'] = df['Name'].str.strip().str.title()
df['Gender'] = df['Gender'].str.upper().str[0] # Normalize to M/F
# Fix: Male→M, Female→F
df['Gender'] = df['Gender'].replace({'M': 'M', 'F': 'F'})

# — Step 3: Handle Invalid Values —
# Age must be between 0 and 120
df.loc[(df['Age'] < 0) | (df['Age'] > 120), 'Age'] = np.nan
df['Age'] = df['Age'].fillna(df['Age'].median())

# — Step 4: Handle Outliers (IQR) —
Q1 = df['Purchase_Amount'].quantile(0.25)
Q3 = df['Purchase_Amount'].quantile(0.75)
IQR = Q3 - Q1
upper_bound = Q3 + 1.5 * IQR
# Cap outliers instead of removing
df['Purchase_Amount'] = df['Purchase_Amount'].clip(upper=upper_bound)
df['Purchase_Amount'] = df['Purchase_Amount'].fillna(df['Purchase_Amount'].median())

# — Step 5: Handle Missing Names —
df['Name'] = df['Name'].fillna('Unknown')

# — Step 6: Validate Email —
df['Valid_Email'] = df['Email'].str.contains(r'^[\w\.-]+@[\w\.-]+\.\w+$', na=False)

# — Step 7: Normalize Purchase_Amount —
min_val = df['Purchase_Amount'].min()
max_val = df['Purchase_Amount'].max()
df['Purchase_Normalized'] = (df['Purchase_Amount'] - min_val) / (max_val - min_val)

print("\nCleaned Dataset:\n", df)
df.to_csv('cleaned_ecommerce.csv', index=False)
```

Pipeline Summary:

1. Deduplication → 2. Text standardization → 3. Invalid value detection → 4. Outlier handling (IQR capping) → 5. Missing value imputation → 6. Validation → 7. Normalization

Q11. The government wants to implement a predictive policing system using historical crime data. Critically discuss the ethical implications, potential biases, and how a data scientist should approach this responsibly.

Ethical Analysis of Predictive Policing:

1. Algorithmic Bias — The Feedback Loop:

- Historical crime data reflects **where police have been deployed**, not where crime actually occurs.
- Over-policed communities (often minorities) have more recorded arrests → model predicts higher crime there → more police deployed → more arrests → **self-reinforcing bias loop**.
- **Example:** PredPol (US) was found to disproportionately target Black and Hispanic neighbourhoods.

2. Privacy Concerns:

- Surveillance data (CCTV, phone records, social media monitoring) raises serious privacy violations.
- Citizens may be profiled and treated as suspects without probable cause.
- Violates Article 21 (Right to Privacy) in India and Fourth Amendment in the US.

3. Lack of Transparency:

- If the algorithm is proprietary, citizens cannot challenge its decisions.
- A person flagged as "high risk" may face consequences without understanding why.
- Violates the principle of **due process**.

4. Discrimination:

- Features like ZIP code, income level, and neighbourhood are **proxies for race/caste**.
- Even without explicitly using protected attributes, the model can discriminate.

5. Accountability Gap:

- When the algorithm makes a wrong prediction leading to unjust arrest, who is responsible? The developer? The police? The vendor?

Responsible Approach:

1. **Audit for bias** — Regularly test model predictions across demographic groups.
2. **Transparency** — Publish methodology, allow independent audits.
3. **Human oversight** — Algorithm should assist, not replace, human judgment.
4. **Community involvement** — Include community representatives in system design.
5. **Fairness constraints** — Apply demographic parity or equalized odds in model training.
6. **Data governance** — Clear policies on data retention, access, and usage.

SECTION C — Long Answer (20 Marks Each, Do Any 2 of 3 = 40 Marks)

Q12. (a) Critically compare all four types of ML with real-world case studies for each (10 Marks). (b) Design a complete ML pipeline for credit card fraud detection, addressing class imbalance, feature engineering, and evaluation metrics in detail (10 Marks).

(a) Four Types of ML — Critical Comparison:

Aspect	Supervised	Unsupervised	Semi-Supervised	Reinforcement
Data	Fully labelled	Unlabelled	Mix (few labelled)	No data (environment)
Learning	From teacher (labels)	Self-discovery	Teacher + self-learning	Trial & error (rewards)
Goal	Predict Y from X	Find structure	Predict with less labels	Maximize reward
Overfitting Risk	High if small dataset	Lower	Moderate	Via exploration-exploitation

Case Studies:

Supervised — Cancer Diagnosis: Trained on labelled pathology slides (malignant/benign). Google's AI achieved 99% accuracy in detecting breast cancer metastasis — outperforming pathologists.

Unsupervised — Customer Segmentation at Spotify: K-Means clustering groups 400M+ users into listening profiles without predefined labels → powers Discover Weekly playlists.

Semi-Supervised — Google Photos Face Recognition: User labels a few faces manually → system propagates labels to thousands of similar unlabelled photos using contrastive learning.

Reinforcement — AlphaFold (DeepMind): Learned protein folding through simulation rewards → solved a 50-year biology problem → won Nobel Prize. Agent gets rewards for correct structure predictions, penalties for deviations from known structures.

(b) Credit Card Fraud Detection Pipeline:

Challenge: Highly imbalanced (99.8% legitimate, 0.2% fraud).

Step 1 — Data Collection:

- Transaction features: amount, merchant category, time, location, device.
- Account features: average spend, account age, past fraud flags.

Step 2 — Feature Engineering:


```
# Time-based features
df['hour'] = df['timestamp'].dt.hour
df['is_night'] = (df['hour'] >= 22) | (df['hour'] <= 5)

# Deviation from user's average
df['amount_deviation'] = (df['amount'] - df.groupby('user_id')['amount'].transform('mean')) / \
    df.groupby('user_id')['amount'].transform('std')

# Velocity features
df['txn_count_1hr'] = df.groupby('user_id')['timestamp'].transform(
    lambda x: x.rolling('1H').count()
)

# Geographic anomaly
df['distance_from_home'] = haversine(df['lat'], df['lon'], df['home_lat'], df['home_lon'])
```

Step 3 — Handling Class Imbalance:

- **SMOTE** to oversample fraud class (from 0.2% to ~10%).
- **Class weights:** `model = RandomForestClassifier(class_weight={0:1, 1:100})`
- **Cost-sensitive learning:** Higher misclassification cost for fraud.

Step 4 — Model Selection:

- Train: Logistic Regression (baseline), Random Forest, XGBoost, Isolation Forest (anomaly detection).
- Use **stratified 5-fold cross-validation**.

Step 5 — Evaluation (NOT accuracy):

Metric	Value	Interpretation
Precision	0.85	85% of flagged transactions are actually fraud
Recall	0.92	92% of actual frauds are caught
F1-Score	0.88	Harmonic mean balances precision & recall
AUC-ROC	0.97	Strong class separability
PR-AUC	0.82	Better metric for imbalanced data than ROC

Recall is prioritized — missing fraud costs the bank money and customer trust.

Step 6 — Deployment:

- Real-time scoring via REST API (< 100ms latency).
- Threshold tuning: flag transactions with fraud probability > 0.3 for review.
- Monitor for data drift (new fraud patterns, seasonal spending changes).

Q13. Write an advanced Python program that performs complete EDA on a complex dataset including correlation analysis, advanced visualization, feature engineering, and statistical hypothesis testing.

File: telecom_churn.csv with columns: CustomerID, Gender, SeniorCitizen, Tenure, PhoneService, InternetService, MonthlyCharges, TotalCharges, Contract, Churn

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# =====
# 1. Load & Inspect
```

```

# =====
df = pd.read_csv('telecom_churn.csv')
print(f"Shape: {df.shape}")
print(f"Dtypes:\n{df.dtypes}")
print(f"\nNull values:\n{df.isnull().sum()}")
print(f"\nTarget distribution:\n{df['Churn'].value_counts(normalize=True).round(3)}")

# =====
# 2. Data Cleaning
# =====
# TotalCharges might have whitespace → convert to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())

# =====
# 3. Descriptive Statistics
# =====
print("\n— Central Tendency —")
for col in ['Tenure', 'MonthlyCharges', 'TotalCharges']:
    print(f"\n{col}:")
    print(f"  Mean  : {df[col].mean():.2f}")
    print(f"  Median: {df[col].median():.2f}")
    print(f"  Mode   : {df[col].mode()[0]:.2f}")
    print(f"  Std    : {df[col].std():.2f}")
    print(f"  Var    : {df[col].var():.2f}")
    print(f"  IQR    : {df[col].quantile(0.75) - df[col].quantile(0.25):.2f}")

# =====
# 4. Feature Engineering
# =====
# Average monthly spend
df['AvgMonthlySpend'] = df['TotalCharges'] / (df['Tenure'] + 1)

# Tenure category
df['Tenure_Group'] = pd.cut(df['Tenure'],
    bins=[0, 12, 24, 48, 72],
    labels=['0-1yr', '1-2yr', '2-4yr', '4-6yr'])

# High spender flag
df['HighSpender'] = (df['MonthlyCharges'] > df['MonthlyCharges'].quantile(0.75)).astype(int)

# =====
# 5. Correlation Analysis
# =====
numeric_cols = df.select_dtypes(include=[np.number])
correlation = numeric_cols.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation, annot=True, cmap='RdBu_r', center=0, fmt='.2f',
    square=True, linewidths=0.5)
plt.title('Feature Correlation Heatmap', fontsize=14)
plt.tight_layout()
plt.show()

# =====
# 6. Advanced Visualizations
# =====

# Subplot: 2x2 dashboard
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

```

```

# 6a. Distribution of MonthlyCharges by Churn
sns.histplot(data=df, x='MonthlyCharges', hue='Churn', kde=True,
             ax=axes[0,0], palette='Set1')
axes[0,0].set_title('Monthly Charges by Churn Status')

# 6b. Boxplot: Tenure by Churn
sns.boxplot(x='Churn', y='Tenure', data=df, palette='Set2', ax=axes[0,1])
axes[0,1].set_title('Tenure Distribution by Churn')

# 6c. Count plot: Contract type vs Churn
sns.countplot(x='Contract', hue='Churn', data=df, palette='Set3', ax=axes[1,0])
axes[1,0].set_title('Contract Type vs Churn')

# 6d. Violin plot: Monthly Charges by Contract
sns.violinplot(x='Contract', y='MonthlyCharges', data=df,
              palette='muted', ax=axes[1,1])
axes[1,1].set_title('Monthly Charges by Contract Type')

plt.tight_layout()
plt.show()

# =====
# 7. Statistical Test – Is mean tenure different for churned vs non-churned?
# =====
churn_yes = df[df['Churn'] == 'Yes']['Tenure']
churn_no = df[df['Churn'] == 'No']['Tenure']

t_stat, p_value = stats.ttest_ind(churn_yes, churn_no)
print(f"\n— T-Test: Tenure by Churn —")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.6f}")
if p_value < 0.05:
    print("Result: SIGNIFICANT – Mean tenure differs between churned and non-churned (p < 0.05)")
else:
    print("Result: NOT significant (p >= 0.05)")

# =====
# 8. Outlier Detection (IQR)
# =====
for col in ['MonthlyCharges', 'TotalCharges']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    print(f"\n{col}: IQR={IQR:.0f}, Bounds=[{lower:.0f}, {upper:.0f}], Outliers={len(outliers)}")

# =====
# 9. Save
# =====
df.to_csv('telecom_processed.csv', index=False)
print("\nProcessed dataset saved!")

```

Q14. (a) Explain the complete Data Science Lifecycle and apply it to a COVID-19 pandemic tracking scenario (10 Marks). (b) Critically discuss how data quality issues (missing data, noise, outliers, inconsistency) propagate through the data pipeline and degrade model performance. Propose a comprehensive data quality framework (10 Marks).

(a) Data Science Lifecycle — COVID-19 Tracking:

Phase	Application to COVID-19
Problem	"Predict hotspot districts and hospital bed requirements for the next 2 weeks."
Collection	RT-PCR test results, hospital admissions, ICMR data, Aadhaar-linked mobility data, Aarogya Setu app.
Cleaning	Duplicate patient entries across states, inconsistent date formats (DD/MM vs MM/DD), missing recovery dates, incorrect pincode mappings.
EDA	Time-series plots of daily cases by state. Heatmaps of district-level infection rates. Age-group distribution of severity. Correlation between population density and case count.
Feature Engineering	7-day rolling average, reproduction number (R_0), vaccination rate per district, mobility index, ICU occupancy rate.
Modelling	SEIR compartmental model for epidemiological simulation. XGBoost for district-level 14-day case prediction. LSTM for time-series forecasting.
Evaluation	RMSE for case predictions. Recall for hotspot identification (must not miss emerging hotspots).
Deployment	Real-time dashboard (like covid19india.org). Automated alerts when $R_0 > 1.0$ in any district. Resource allocation recommendations for hospital beds and oxygen.

(b) Data Quality Issues and Propagation:

The Data Quality Cascade:

Poor Data Quality → Flawed EDA → Wrong Features → Bad Model → Wrong Decisions

1. Missing Data Propagation:

- Many ML algorithms cannot handle NaN → rows dropped → reduced sample size.
- Imputation with wrong method (e.g., mean on skewed data) introduces bias.
- Impact:** Model learns distorted distributions, undermining generalization.

2. Noise Propagation:

- Random errors in data (typos, sensor glitches) confuse the model.
- Model fits to noise instead of signal → overfitting.
- Impact:** Poor performance on unseen data, high variance in predictions.

3. Outlier Propagation:

- Outliers distort mean, variance, and correlation calculations.
- Distance-based algorithms (k-NN, K-Means) are highly sensitive.
- Linear regression shifts dramatically toward outliers.
- Impact:** Biased model parameters, incorrect cluster assignments.

4. Inconsistency Propagation:

- "Male", "M", "male" treated as different categories → inflated feature space.
- Date format inconsistencies → incorrect temporal ordering.
- Impact:** Wrong one-hot encoding dimensions, broken time-series analysis.

Comprehensive Data Quality Framework:

Layer	Activity	Tools/Techniques
1. Profiling	Understand data distributions, types, cardinality.	Pandas Profiling, Great Expectations.
2. Validation	Define rules (Age ∈ [0,120], Email matches regex).	Schema validation, assert statements.
3. Cleaning	Fix issues: impute, deduplicate, standardize.	Pandas, OpenRefine.
4. Monitoring	Continuous checks in production.	Data drift detection (KS test, PSI).
5. Documentation	Record all transformations for reproducibility.	Data dictionaries, lineage tracking.

6. Governance	Define ownership, access controls, retention policies.	Role-based access, audit logs.
---------------	--	--------------------------------